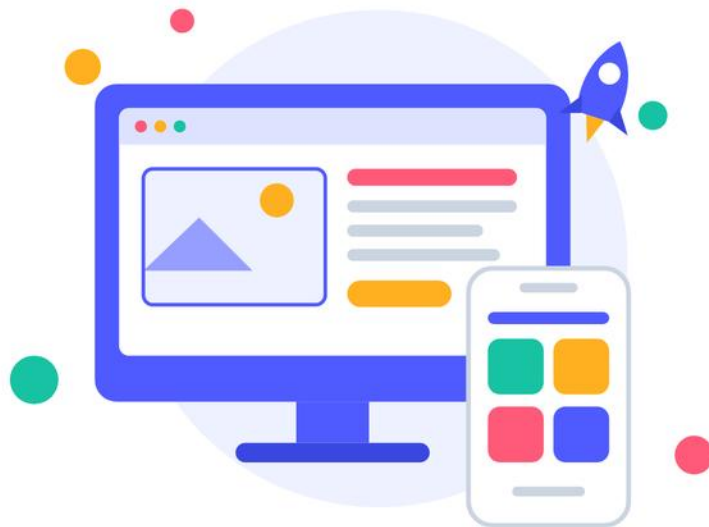


WEEK 02 · REFERENCE

FORMS, TABLES, AND THE READER WHO CANNOT SEE — THE CATALOGUE

Every input type, get or post, every datetime format, how a field gets its name, ARIA's five rules, the WCAG rows, and who sees what. Optional depth, not examined.

YOSSI ARVIV · FRONT-END WEB DEVELOPMENT · RUPPIN



Even this week was typed in a cycle. This deck is where you go to know why.

FORMS — THE INPUT TYPES WORTH KNOWING

text	anything short
email	validates the shape, changes the phone keyboard
tel	numeric keypad; no validation
number	with <code>min</code> , <code>max</code> , <code>step</code>
date	a real date picker (also <code>time</code> , <code>month</code>)
password · url	masked, not secure; a URL shape check
checkbox · radio	many yes/no · one of several, same name (also <code>file</code> , <code>range</code> , <code>color</code>)
textarea · select	elements, not types: multi-line; a list of options

Create Account

Full name

Email

Newsletter topics

☐ Tech

☐ Deals

Plan

☒ Basic

☐ Pro

Country

About you

Every field here has a real label. That is the assignment.

THE FORM, DONE PROPERLY

```
1 <form method="get">
2   <fieldset>
3     <legend>Your details</legend>
4
5     <label for="name">Full name</label>
6     <input type="text" id="name" name="name" required />
7
8     <label for="email">Email</label>
9     <input type="email" id="email" name="email" required />
10
11    <button type="submit">Send</button>
12  </fieldset>
13 </form>
```

No `action` means "send to this page" — with `get`, the fields land in the address bar, which is why the cycles use it as a zero-JavaScript debugger. `action="/contact"` and a server arrive in week 11. `button` always carries `type`: the validator (`no-implicit-button-type`) insists.

ID, NAME, FOR — THREE ATTRIBUTES, THREE JOBS

```
1 <label for="email">Email</label>
2 <input type="email" id="email" name="email" />
3
4 <input type="radio" id="s" name="member" value="student" checked />
5 <input type="radio" id="t" name="member" value="staff" />
6
7 <select id="topic" name="topic">
8   <option value="course">This course</option>
9 </select>
```

name

?

email=maya%40example.com&member=student&topic=course

value

For the page: unique; what for, href="#..." and aria-describedby point at.

For the server: the key in the query string. Radios share it; a select sends the chosen option's value.

What is label for only, you check a bar or opt, on the text for the radio field:
<label>Email <input></label>.

REQUIRED, AND THE BROWSER'S OWN VALIDATION

```
<input type="email" required />
<input type="number" min="1" max="4" step="1" />
<input type="text" minlength="2" maxlength="40" />
<input type="text" pattern="[0-9]{9}" title="nine digits" />
<form novalidate>...</form>
```

required

An empty Send stops on the field — no JavaScript.

type

`email` refuses text without `@`;
`number` refuses letters.

limits

`min`, `max`, `step`, `minlength`,
`maxlength`, `pattern` (a
regex).

novalidate

On the `form`: switches it all off.
Never as a way out.

The browser validates for the user; the server validates again for you (week 11).

GET OR POST

METHOD="GET"

The fields travel **in the URL**: `?q=alien&room=204`. For requests that *read* — search, filter, page 2. The result can be bookmarked, shared, refreshed, and it sits in the history. This week's debugger: Send, then read the address bar.

METHOD="POST"

The fields travel **in the request body**, not in the URL. For requests that *change* something — sign up, send a message, pay. Not bookmarkable, and the browser asks before re-sending it on refresh. Needs a server to receive it: week 11.

Neither is "more secure": both are readable in transit without TLS. The difference is meaning — read versus change — and where the data shows up.

TABLES — THE FULL ANATOMY

```
1 <table>
2   <caption>Hours</caption>
3   <thead>
4     <tr>
5       <th scope="col">Day</th>
6       <th scope="col">Hours</th>
7     </tr>
8   </thead>
9   <tbody>
10    <tr>
11      <th scope="row">Sunday</th>
12      <td>08:00–16:00</td>
13    </tr>
14  </tbody>
15 </table>
```

`scope` says which header owns which cell: `col`, `row`, `colgroup`, `rowgroup`. Also `tfoot`; `colspan` / `rowspan`; `headers="id id"` for tangled tables. Rows go inside `tbody`.

DATETIME — EVERY VALID FORMAT

DATETIME	WHAT IT IS
<code>2027-11-02</code>	a date — year-month-day, two digits each
<code>2027-11 · 2027 · 11-02</code>	a month · a year · a yearless date (a birthday)
<code>19:30 · 19:30:00</code>	a time
<code>2027-11-02T19:30</code>	a local date and time (a space instead of <code>T</code> is also valid)
<code>2027-11-02T17:30Z · ...T19:30+02:00</code>	a global date and time, with the offset from UTC
<code>2027-W44 · PT2H30M</code>	a week · a duration

Not a format: `2 Nov 2027`, `02/11/2027`, `Tuesday`. Nothing complains and no machine can read it — that text goes inside the element. Source: the HTML Living Standard, "the `time` element".

DETAILS AND SUMMARY – THE ATTRIBUTES

```
1 <details open>
2   <summary>How screenings work</summary>
3   <p>Doors open at 19:15.</p>
4 </details>
5 <details name="faq">...</details>
```

open

Present: expanded. The browser toggles it itself; write it to start open.

summary

First child. Omit it and the browser shows "Details". Focusable; Space or Enter toggles.

name

Same name details all a group: with opening of it closes the others (as only the summary (2014) closure triangle). The hidden content is not in the tree until it opens.

PICTURE VS SRCSET AND SIZES

ART DIRECTION – A DIFFERENT PICTURE PER WIDTH

```
1 <picture>
2   <source media="(min-width: 700px)" srcset="wide.svg" width="320" height="180" />
3   <source type="image/avif" srcset="narrow.avif" />
4   
5 </picture>
```

RESOLUTION SWITCHING – THE SAME PICTURE, MORE PIXELS

```
1 
```

`source` s first, in order; the first whose `media` matches the browser knows wins. `img` last, always: it is the fallback, the owner of `alt`.

No picture needed. `srcset` lists candidates with the file names; `sizes` tells the browser how wide the image will be laid out, and it picks the smallest candidate that is sharp enough for the screen's pixel density. Weeks 5 and 6.

THE ACCESSIBILITY TREE, AND WHERE A NAME COMES FROM

Beside the DOM the browser builds a second tree for assistive technology: every node has a **role**, a **name** (what is announced) and sometimes a **description**. A field's name, in order:

- 1 · **labelledby** `aria-labelledby`: the text of the elements it points at
- 2 · **aria-label** the attribute's string — invisible on screen
- 3 · **label** `label` for or a wrapping `label` — the visible words
- 4 · **title** then `placeholder`: last resorts — why axe passes a placeholder-only field

WHY THE ORDER MATTERS

Higher wins. An `aria-label` on a field with a visible label *replaces* the words in the tree: the user sees one thing, hears another. The description (`aria-describedby`) is read after the name.

Source: Accessible Name and Description Computation; HTML-AAM.

ARIA'S FIVE RULES

1. **Do not use ARIA** if a native element already has the meaning and the behaviour: a `button`, not `<div role="button">`; a `label`, not `aria-label`.
2. **Do not change native semantics** unless you really have to.
3. **Every interactive ARIA control must work with the keyboard.** A `div` with a role gets no Enter, Space or focus for free.
4. **Never hide something focusable** with `role="presentation"` or `aria-hidden="true"`.
5. **Every interactive element needs an accessible name.**

This week's three ARIA attributes are all additions to native elements:

`aria-label` on a `nav`, `aria-describedby` for a hint, `aria-current` for the page. Source: W3C, "Using ARIA".



WCAG 2.2 – THE ROWS THIS WEEK TOUCHES

CRITERION	LEVEL	WHAT YOU TYPED FOR IT
1.1.1 NON-TEXT CONTENT	A	<code>alt</code> on every <code>img</code> , including the one in a <code>picture</code>
1.3.1 INFO AND RELATIONSHIPS	A	<code>label</code> for, <code>fieldset/legend</code> , <code>th scope</code> , <code>caption</code> , headings
2.1.1 KEYBOARD · 2.4.1 BYPASS BLOCKS · 2.4.3 FOCUS ORDER	A	everything reachable by Tab, in document order; the skip link; no positive <code>tabindex</code>
2.4.6 HEADINGS AND LABELS · 2.4.7 FOCUS VISIBLE	AA	headings and labels that say what the thing is; the ring on <code>main</code>
2.5.8 TARGET SIZE (MINIMUM)	AA	24×24 CSS px or spacing — new in 2.2; <code>target-size</code> ; needs CSS
3.3.2 LABELS OR INSTRUCTIONS	A	a label on every field; the hint with <code>aria-describedby</code>
4.1.2 NAME, ROLE, VALUE	A	every control has a name; <code>aria-current</code> as state

4.1.1 Parsing was removed in 2.2: the validator is the course's rule, not

THE AXE RULES YOU SAW THIS WEEK

RULE ID	IMPACT · TAG	WHAT TRIGGERED IT
LABEL	critical · wcag2a	a field with no name at all (the AI-cycle textarea; Lighthouse 91)
IMAGE-ALT	critical · wcag2a	an <code>img</code> without <code>alt</code>
HEADING-ORDER	moderate · best-practice	<code>h2</code> then <code>h4</code> (cycle 3 start, Lighthouse 93)
TABINDEX	serious · best-practice	<code>tabindex="1"</code> on the Send button (Lighthouse 91)
LABEL-TITLE-ONLY	serious · best-practice	a field whose only name is its <code>title</code>
LANDMARK-ONE-MAIN · REGION	moderate · best-practice	the bare demo form with no <code>main</code>
TARGET-SIZE	serious · wcag22aa	nav links with no spacing — every CSS-less page

The grader runs axe-core with `wcag2a` and `wcag2aa`, serious and critical only — so `heading-order` and `tabindex` get their own rows. Measured: axe-core 4.13, Lighthouse 12.8; the placeholder-only form scores 100.

THE SKIP LINK, LINE BY LINE

```
1 <body>
2 <a href="#main">Skip to content</a>
3 <header>...</header>
4 <nav>...</nav>
5 <main id="main" tabindex="-1">...</main>
```

first

First in `body`, so the first Tab reaches it.

href

A fragment link: the target is an `id` on the same page.

tabindex

With no CSS with the first line of content is a page if on the 3rd screen, without focus stays over `display: none` m still continues from `main`, but assistive technology follows focus.

ARIA-CURRENT — THE VALUES

page	the link to the page you are on — this week's value, exactly one per page
step	the current step of a wizard or a checkout
location	the current place in a breadcrumb or a map
date	today in a calendar
time	the current slot in a timetable
true · false	current, with no more specific meaning; absent

ONE FACT, ONE PLACE

The attribute is the truth: a screen reader announces "current page", and next week `nav a[aria-current="page"]` is the selector that styles it. A `class="active"` would be a second copy of the same fact to keep in sync. Source: WAI-ARIA 1.2, `aria-current`.

HEADINGS ARE AN OUTLINE

WHAT A SCREEN READER LIST

```
<h1>Ruppin Film Club</h1>
  <h2>Screenings this month</h2>
  <h2>Join the club</h2>
  <h2>About the club</h2>
    <h3>Where we meet</h3>
```

one h1

The page's title. The assignment grades exactly one.

no skipped level

After `h2` comes `h3`. `h4` "because `h3` looked too big" is the heading-order row.

no font size

Every section and article of gets a heading, aside to off it (has content bits, WDA's H key). Size is CSS, next week.

WHERE THE TREE LIVES IN DEVTOOLS

ELEMENTS · ACCESSIBILITY PANE

Select any element. The side pane (next to Styles, Computed) shows its **Name**, **Role**, **Description**, the ARIA attributes on it, and the computed properties — and the path of nodes above it.

THE FULL TREE

The accessibility icon at the top-right of the DOM tree switches the whole Elements panel to the accessibility tree: roles and names instead of tags. `columnheader`, `group`, `textbox "Email"` — this is what the reader gets.

LIGHTHOUSE

The Lighthouse tab · tick Accessibility only · Analyze page load. Each red row names the rule and the element; "Learn more" is the axe rule's page. Free, no extension. The axe DevTools extension shows the same rules with impact levels.

In the console: `document.activeElement` tells you where focus is; `$$('nav a[aria-current="page"]').length` counts what the reader will hear as current.

THE VALIDATOR, AXE, AND A CLICK — WHO SEES WHAT

THE MISTAKE	HTML-VVALIDATE 9	AXE / LIGHTHOUSE	THE BROWSER, BY HAND
DUPLICATE ID	<code>no-dup-id</code> , <code>form-dup-name</code>	silent	click Email — focus lands in Full name
FOR = THE NAME	silent	<code>label</code> critical	click — nothing
PLACEHOLDER ONLY · ARIA-LABEL	silent	silent (100)	click — nothing; typing hides the hint
BOLD TDS AS HEADERS	silent	silent	the tree says <code>cell</code> , not <code>columnheader</code>
H2 THEN H4	silent	<code>heading-order</code>	a hole in the headings list
TABINDEX="1"	silent	<code>tabindex</code> serious	first Tab lands on Send
SOURCE AFTER IMG	<code>element-permitted-order</code>	silent	the narrow image at every width
DATETIME="2 NOV 2027"	silent	silent	the machine reads nothing

Every row measured. No single tool sees the whole column — so the self-check lists things that *happen*, not things you wrote.

ACCESSIBILITY IS GRADED FROM THIS WEEK

Keyboard

Tab through the whole page. Everything interactive must be reachable, in document order.

Focus

You must be able to see where you are. Never delete the focus ring.

Landmarks

`header` `nav` `main` `footer` — how a screen-reader user skips to the content; the skip link is the shortcut to `main`.

Names

Every image, every field, every link says what it is — in words that are on the screen.

Every assignment from this one onwards has a row for this: axe with the WCAG A/AA tags, zero serious or critical. Green is the floor; the rows above it are the standard.

